

The pricing engine as a hosted service

2026-09-30 · a short note, separate from the weekly report on purpose

In September you asked whether the code could live somewhere **everyone in the group can run and test it**, rather than only on our machines. In the 16 September trial we answered the first half — the engine runs unchanged on Azure. **This is the second half: it is now a web service, and it has been verified.**

1. What exists

A single HTTPS endpoint on Azure App Service. Send one bond as JSON, get its price, spread and risk back as JSON:

```
POST /price    one bond in, one complete answer out
GET  /health   can this instance actually price, or is it merely running?
```

This is **the same contract the Excel bridge already uses** — the one we built in August, where a spreadsheet sends one bond and receives one answer. A browser page and a spreadsheet are simply two callers of the same door. ★ **A second front end therefore costs no engine work**, which is the part that matters for the JavaScript architecture you demonstrated: the page replaces where the request is *assembled*, not what it means.

It is currently stopped, deliberately — nobody is calling it yet, and a stopped service cannot be reached by anyone. One command brings it back in about a minute.

2. How we know it is right, not just running

This is the part worth a paragraph, because "the website loads" is not the same claim as "it produces the right numbers".

We ship **11 reference cases** — a plain bond, a callable, a puttable, a sinking-fund bond, two floating-rate notes, and five deliberate error cases — each with its answer recorded in the repository. A single command sends all 11 to the deployed service and compares every field of every reply against that record:

```
PASS  the deployed service reproduces the record: 11 fixtures, 156 numbers,
      worst 0.00% of tolerance.
```

★ **156 numbers, and not one of them moved.** Every text field, error code and field name identical; every number bit-for-bit. The check reports how many values it compared and **refuses to report success if it compared too few** — a green result that quietly examined nothing is worse than a red one.

★ It needs nothing installed beyond Python itself, so **anyone can run it against the service** — you, the cloud team, or an automated check — without a working copy of the pricing environment.

3. How it is deployed

One script in the repository, not a sequence of portal clicks.

```
bash deploy/azure/deploy.sh          # infrastructure, settings, code and data
python scripts/remote_smoke.py <url> # the verification above
```

Every value it sets carries a comment saying why. ★ **This matters more than it sounds.** A cloud setup that cannot be reproduced from a file tends to stop working as unaccountably as it started; one that is a script can be rebuilt from scratch in about five minutes, moved to a different subscription, or handed to your cloud team to run themselves.

The service: Linux App Service, Python 3.12, two worker processes. Two workers are two independent processes, which is exactly the parallel execution the browser architecture wants — **N bonds priced at once needs nothing from us**, because we made one bond the unit of work back in August.

The data (curve files and the portfolio, ~34 MB) ships inside the deployment package, so the code and the data it was tested against are locked together and cannot drift apart.

4. Cost

About USD 13 per month, on the card registered to the account. That is the smallest tier that keeps the service warm; the free tier unloads it after twenty minutes idle, which would make the first request of the day take a minute and look broken.

⚠ **Stopping the app does not stop this charge** — the hosting plan bills whether the app runs or not. Since the whole deployment is a script that rebuilds in five minutes, it is entirely reasonable to delete it between demonstrations and stand it up when needed. **Tell us which you prefer**; we have left it stopped and billing, ready to start.

5. What is deliberately not done yet

Sign-in. Access is currently restricted to a single network address. Proper sign-in — where you add a colleague to a list and they can use the service — is written and ready to switch on, and we have held it back on purpose: ★ **a browser page and a machine-to-machine caller need different sign-in flows**, and choosing before your JavaScript front end is specified means choosing twice. **The trigger is the first person who is not us.**

Cross-origin access. A browser calling this from a different web page will be refused **by the browser itself, before the request ever reaches the server** — so nothing appears in any log and it looks like the service is down. It is a one-line setting, but it needs to name the page it is allowing, which is a decision that belongs with the front-end design.

★ **Both are the same conversation**, and both wait on the same thing: where the browser page is going to live.